Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. doi: 10.18637/jss.v000.i00

capnet: Contribution-Capped Regularization for Elastic Net Models in R

Abstract

This paper introduces **capnet**, an R package implementing contribution-capped regularization, a new penalization and constraint framework for generalized linear regression models. **capnet** extends elastic net by imposing feature-specific caps on coefficient contributions, enabling structured shrinkage, easy interpretability, and explicit control of feature influence. The method is motivated by applications in quantitative finance, risk management, and high-dimensional prediction tasks where no existing package provides comparable constraint functionality. We present the statistical formulation, optimization algorithm, software architecture, and empirical performance through simulation and real-world case studies.

Keywords: regularization, constrained optimization, elastic net, R.

1. Introduction

Penalized regression methods are a central component of modern statistical modeling. Approaches such as ridge regression (Hoerl and Kennard 1970), the lasso (Tibshirani 1996), and the elastic net (Zou and Hastie 2005) control model complexity by shrinking regression coefficients, improving stability and predictive performance in high-dimensional or ill-conditioned settings. When combined with generalized linear models (GLMs) (McCullagh and Nelder 1989), these methods provide flexible and widely used tools across statistics, econometrics, and machine learning.

Despite their success, existing penalization frameworks operate exclusively on the coefficient vector and do not directly regulate the realized influence of predictors on model outputs. For observation i and predictor j, the quantity $X_{ij}\beta_j$ represents the actual contribution of predictor j to the linear predictor. In many applications, particularly those involving heavy-tailed predictors or evaluation across different regimes, it is this realized contribution, instead of coefficient magnitude, that determines model behavior. Even when coefficients are heavily

regularized, extreme predictor realizations can induce disproportionately large contributions, leading to unstable predictions, excessive sensitivity to rare events, or violations of domain-specific constraints.

Applications in which bounding predictor influence is essential arise in numerous domains. The original motivation for **capnet** stems from quantitative finance, where the framework was developed for a market-timing model. In that setting, predictors derived from volatility measures or other tail-sensitive market indicators exhibit heavy-tailed behavior and occasionally attain extreme values. Although such regimes may represent a small fraction of the training data, they are central to predictive evaluation and risk management. In these cases, extreme realizations can induce sharp swings in individual predictor contributions, translating into concentration risk both in statistical estimation and in downstream portfolio decisions. Directly regulating realized contributions provides a principled means of limiting such effects.

The same logic extends naturally to other disciplines. In risk assessment, credit scoring, epidemiology, and related fields, predictors often display skewed or heavy-tailed distributions, and predictions are frequently evaluated under stress scenarios rather than average conditions. In such settings, it may be undesirable for any single predictor to induce large changes in predicted probabilities or expected outcomes solely due to rare but extreme covariate values. Existing penalized regression frameworks do not offer a mechanism for enforcing such constraints, as they operate exclusively through shrinkage of the coefficient vector rather than through regulation of realized predictor contributions.

Existing approaches address instability arising from extreme observations through a variety of mechanisms, but none directly regulate predictor-level contributions. Classical penalization methods such as ridge regression, the lasso, and the elastic net constrain the magnitude of the coefficient vector, thereby controlling overall model complexity. Robust regression techniques mitigate the influence of extreme observations by downweighting residuals, focusing on stability in the response space. Distributionally robust optimization methods extend this idea by guarding against worst-case distributional shifts in the data-generating process. While these approaches target different aspects of model instability, they share a common limitation: they do not directly constrain the realized contributions $X_{ij}\beta_j$ of individual predictors. As a result, even well-regularized or robust models may produce predictions dominated by a small number of predictors when covariates take extreme values.

A number of existing software packages provide penalized or constrained GLM fitting, but none directly address predictor contributions. The **glmnet** package (Friedman *et al.* 2010) implements efficient coordinate-descent algorithms for ℓ_1- and ℓ_2-regularized GLMs, but regularization acts only on the coefficients and therefore does not constrain the realized quantities $X_{ij}\beta_j$. Packages such as **mgcv** (Wood 2017) impose smoothness or shape constraints, which control functional form rather than bounding predictor influence. Bias-reduction methods for GLMs, including **brglm2** (Kosmidis and Firth 2019), address small-sample bias and separation but do not regulate individual predictor effects. To our knowledge, no existing framework for penalized GLMs incorporates penalties that directly constrain feature contributions.

Several ad hoc strategies are sometimes employed in practice to mitigate large or unstable predictor effects, though these approaches have important limitations. Winsorizing or trimming predictors caps raw covariate values independently of the fitted model, discarding tail information and requiring arbitrary threshold choices. Pre-filtering predictors based on variance or range removes extreme features outright rather than controlling their influence, often

sacrificing predictive signal. Post-hoc capping of fitted values modifies predictions without altering the underlying coefficients, breaking the coherence of the fitted model and complicating interpretation and inference. By contrast, **capnet** embeds contribution control directly into the estimation procedure, ensuring that the fitted model itself respects constraints on predictor influence across the regimes of interest.

This paper introduces **capnet**, a penalized regression framework and accompanying R package that directly constrains predictor contributions. **capnet** extends the elastic net by imposing smooth, feature-specific caps on the realized quantities $X_{ij}\beta_j$ through a differentiable hinge-squared penalty. This formulation allows users to limit the influence of individual predictors while preserving flexibility in model estimation. A key feature of the framework is the ability to specify an evaluation matrix distinct from the training data, enabling contribution constraints to be enforced not only in-sample but also in user-defined scenarios, such as stress conditions or anticipated deployment environments.

The contribution-cap formulation integrates naturally with generalized linear models. The resulting optimization problem remains convex and can be efficiently solved using limited-memory quasi-Newton methods, with automatic handling of ℓ_1 regularization through orthant-wise updates. The **capnet** package provides a practical and extensible implementation, including functionality for model fitting, cross-validation, coefficient path analysis, and sequential (walk-forward) estimation. The interface is designed to be consistent with established tools for penalized regression, facilitating adoption in existing workflows.

The primary contribution of this work is conceptual as well as computational. **capnet** introduces a new dimension of regularization by shifting the focus from controlling parameter magnitude to controlling feature-level influence. This perspective is particularly relevant in applications such as quantitative finance, risk management, credit modeling, and epidemiology, where extreme covariate realizations and regime-specific evaluation criteria play a central role. By embedding contribution control directly into the estimation procedure, **capnet** aligns statistical modeling with practical constraints on predictor behavior.

The remainder of this article is organized as follows. Section 2 discusses the practical motivations underlying contribution caps and reviews related approaches for handling heavy-tailed predictors. Section 3 presents the contribution-cap framework and its integration with generalized linear models. Section 4 describes the software implementation and optimization strategy used in the **capnet** package. Section 5 provides empirical examples illustrating the use of **capnet** in Gaussian and GLM settings. Section 6 concludes with a discussion of limitations and directions for future research.

2. Motivations

2.1. Theoretical Motivations

Contribution caps are motivated by predictive settings in which a small number of predictors can exert disproportionate influence on fitted values, particularly in regions of the covariate space that are rare but substantively important. Consider a predictive task governed by the data generating process shown in Figure 1, where predictors $X_1, \ldots, X_p$ jointly determine the response Y.

In an idealized setting, where $X \in \mathbb{R}^{n \times p}$ is fully observed, $n \gg p$, the functional form is linear

and additive, and classical assumptions hold, ordinary least squares yields the best linear unbiased estimator (BLUE), and no penalization is required. In practice, deviations from this setting—such as multicollinearity, high-dimensionality, or model misspecification—motivate the use of ℓ_1 and ℓ_2 regularization to stabilize estimation.

The setting motivating contribution caps is different. In many applications, predictors exhibit heavy-tailed or highly heterogeneous behavior, and predictive performance is not evaluated uniformly across the covariate space but instead within specific regions of interest. These regions may constitute only a small fraction of the training data yet dominate the practical objective—for example, periods of market stress in finance, extreme environmental conditions in climate modeling, or rare risk profiles in insurance and credit scoring. In such settings, extreme covariate realizations can induce large realized contributions $X_{ij}\beta_j$, even when coefficients are heavily regularized. This creates a form of instability that is not captured by traditional regularization: a model may appear well-behaved in-sample, yet produce predictions dominated by a small number of predictors when evaluated in these regions. The root cause is that classical methods control coefficient magnitude rather than the realized influence of predictors.

A substantial literature on fat-tailed data addresses related issues indirectly by focusing on heavy-tailed response variables rather than heavy-tailed predictors. Common approaches include transforming the response, adopting generalized linear models with heavy-tailed distributions (e.g., Gamma or inverse Gaussian), using generalized distribution regressions, quantile regression, mixture models, or spliced distributions (Shi 2014; Gan and Valdez 2018). These methods improve modeling of the conditional distribution of Y but leave the mapping from predictors to the linear predictor unchanged. As a result, they do not regulate predictor contributions and do not prevent extreme covariate realizations from dominating fitted values.

2.2. Related Approaches

Practitioners have proposed a variety of strategies to mitigate the effects of heavy-tailed predictors in regression models. These approaches include preprocessing techniques such as transformations and winsorization, robust regression and leverage-based methods, and post-hoc modifications to fitted values. While each of these methods addresses certain aspects of instability induced by extreme covariate realizations, they do so indirectly and do not provide explicit control over the realized contributions of predictors to the linear predictor.

Transformations such as logarithmic, power, or rank-based mappings, as well as winsorization and trimming, are commonly used to compress the tails of predictor distributions prior to model fitting. These methods can reduce the influence of extreme observations in-sample and improve numerical stability. However, they operate independently of the fitted model and require heuristic choices of transformation or truncation thresholds. Moreover, they alter the scale and interpretation of predictors and provide no guarantees against extreme predictor realizations encountered in new or stress scenarios.

Robust regression techniques, including M-estimators, reduce the influence of extreme observations by downweighting points with large residuals or high leverage, thereby stabilizing coefficient estimation under heavy-tailed noise (Huber 1973). Relatedly, distributionally robust optimization (DRO) methods introduce robustness by minimizing worst-case expected loss over a neighborhood of plausible data-generating distributions (Ben-Tal *et al.* 2009). In both cases, robustness is achieved through modifications to the loss or weighting scheme and

operates primarily at the level of residuals or aggregate predictive risk. However, neither framework directly regulates the realized contributions of individual predictors to the linear predictor, and high-leverage covariate realizations with small residuals may still exert disproportionate influence. Consequently, even models estimated using robust regression or DRO can produce large predictor contributions in new or stress scenarios, as these methods do not impose explicit bounds on $X_{ij}\beta_j$.

Finally, practitioners may cap the linear predictor or fitted values to prevent extreme predictions. While this approach enforces bounds on outputs, it disconnects the estimation procedure from the predictions ultimately used in practice, as the model is not estimated under the same constraints that govern its deployment. This mismatch can lead to suboptimal coefficient estimates and reduced predictive performance.

Taken together, these approaches can be understood as targeting different aspects of model instability. Preprocessing methods modify the distribution of predictors, robust regression methods control the influence of observations through residuals, and DRO methods address uncertainty in the data-generating process. However, none of these approaches directly constrain predictor-level contributions, leaving open the possibility that individual predictors may dominate model outputs when covariates take extreme values.

Contribution caps address this gap by integrating influence constraints directly into the estimation problem. Rather than modifying predictors ex ante or predictions ex post, the cap penalty operates on realized contributions $Z_{ij}\beta_j$ during optimization. This ensures that predictor influence is bounded in the regions of the covariate space encoded by the evaluation matrix Z, while preserving the probabilistic structure of the underlying GLM. By allowing Z to differ from the training design matrix, contribution caps provide a principled mechanism for enforcing stability not only on observed data but also on stress scenarios or anticipated deployment environments. Importantly, this approach does not rely on fitting separate models across regimes. Instead, capnet incorporates regime-specific considerations within a single estimation problem by enforcing contribution constraints over the evaluation matrix Z. The resulting model adapts its effective behavior across different regions of the covariate space through the optimization objective, while maintaining a unified and interpretable parameterization.

2.3. Motivating Example

In this subsection, we present two simplified linear regression examples that illustrate the theoretical considerations discussed above and demonstrate practical use cases for **capnet**.

First, we consider a simulated setting in which the outcome variable y is generated from a latent feature matrix Z according to a linear data generating process. In estimation and prediction, however, the analyst does not observe Z directly but instead observes a noisy proxy matrix X, whose distribution exhibits heavier tails for a subset of predictors. This setup can be interpreted as a measurement error problem in which the observed covariates are imperfect and heteroskedastic proxies for the true latent features.

Formally, the outcome is generated as

$$y_i = Z_i\beta + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

where β is a sparse coefficient vector.[1]

[1] Sparsity is introduced by randomly setting a subset of coefficients to zero.

The observed covariates are constructed by adding noise to Z. Features are partitioned into a stable set $\mathcal{S}$ and an unstable set $\mathcal{U}$. For features in $\mathcal{S}$, the added noise is homoskedastic Gaussian:

$$\eta_{ij} \sim \mathcal{N}(0, \sigma_{\text{low}}^2), \quad j \in \mathcal{S}.$$

For features in $\mathcal{U}$, the noise variance varies across observations, mimicking leptokurtic measurement error. Specifically, for each observation i and feature $j \in \mathcal{U}$, we draw

$$F_{ij} \sim \text{Bernoulli}(p),$$

and define

$$s_{ij} = \begin{cases} \sigma_{\text{low}}, & F_{ij} = 0, \\ \sigma_{\text{high}}, & F_{ij} = 1. \end{cases}$$

Conditional on F_{ij}, the noise term follows

$$\eta_{ij} \mid F_{ij} \sim \mathcal{N}(0, s_{ij}^2).$$

The observed design matrix is then given by $X = Z + \eta$.

This data generating process produces covariates that are heteroskedastic and heavy-tailed for unstable features. When estimating linear models using X, conventional methods may encounter two related challenges. First, extreme realizations can exert disproportionate influence on coefficient estimates, leading to instability and degraded out-of-sample performance. Second, large covariate realizations may induce excessive concentration of predictive weight in a small number of features, allowing individual predictors to dominate the magnitude and direction of predictions. Importantly, the second issue persists even when coefficient shrinkage is applied. Because classical regularization methods act on β rather than on realized contributions $X_{ij}\beta_j$, extreme covariate realizations can still generate large contributions and dominate model outputs.

In settings where Gaussian assumptions for covariates are appropriate, such extreme observations may be treated as outliers and addressed using winsorization or robust loss functions. However, when the covariate distribution is genuinely heavy-tailed or when it is difficult to distinguish informative extreme values from noise, these approaches may systematically downweight predictive features and degrade performance. By contrast, **capnet** allows features to retain predictive influence while explicitly limiting their contribution to the fitted values, enabling control over feature dominance without discarding extreme observations.

To evaluate performance, the data are split into training and testing sets. The elastic net tuning parameter λ is selected via cross-validation with an equal mixture of ℓ_1 and ℓ_2 penalties. For **capnet**, we consider multiple configurations: (i) global evaluation on the full test matrix versus walk-forward evaluation in which models are refit sequentially, (ii) a range of penalty strengths, and (iii) varying contribution ceiling levels.[2]

Each configuration is compared against the unconstrained elastic net benchmark, corresponding to a zero contribution cap. Tables 1 and 2 report out-of-sample performance for the full and walk-forward evaluation schemes, respectively.

[2]Contribution ceilings are computed as multiples of the univariate regression coefficients between each feature and the outcome in the training set.

		Contribution Ceiling Multiplier				
		0	2	4	8	16
Penalty Strength	0	8.650				
	0.1		8.536	8.441	8.571	8.650
	1		9.071	8.353	8.527	8.650
	10		9.650	8.341	8.519	8.650
	100		9.762	8.340	8.519	8.650

Table 1: Out-of-sample mean squared error (MSE) for capnet models on the simulated dataset. The case $\gamma = 0$ and multiplier $= 0$ corresponds to the uncapped elastic net benchmark. Moderate contribution ceilings (e.g., multipliers of 2–4) improve predictive performance relative to the uncapped model.

		Contribution Ceiling Multiplier				
		0	2	4	8	16
Penalty Strength	0	8.650				
	0.1		8.098	8.104	8.412	8.650
	1		8.118	8.096	8.405	8.650
	10		8.124	8.095	8.404	8.650
	100		8.124	8.095	8.404	8.650

Table 2: Out-of-sample mean squared error (MSE) of walk-forward **capnet** models on the simulated dataset. Each model is refit sequentially along the evaluation path. All capped specifications outperform the uncapped benchmark, illustrating that local contribution control improves predictive stability and accuracy relative to global constraints.

Across configurations, **capnet** frequently outperforms the benchmark model, although excessively restrictive ceilings can degrade performance. Conversely, when ceilings are sufficiently large, the constraint becomes inactive and the solution reduces to the unconstrained elastic net. While the penalty strength affects performance, particularly when contribution ceilings are tight, its influence is generally smaller than that of the ceiling level itself, highlighting the importance of selecting appropriate contribution limits. The walk-forward evaluation approach consistently delivers improved performance and greater stability. This behavior reflects the fact that imposing global contribution constraints over an entire evaluation set can lead to overly aggressive penalization, whereas local, observation-level constraints adapt more flexibly to changing covariate realizations. As discussed further in the software implementation section, the walk-forward approach is generally preferred despite its higher computational cost, particularly when heuristics for selecting contribution ceilings are imperfect.

Second, consider an application in quantitative finance. In a medium-frequency market-timing model, a set of predictors is used to forecast excess equity risk premia. Financial predictors are often noisy and heavy-tailed, making it difficult to distinguish meaningful extreme values from noise. A central challenge is to construct a model that captures informative deviations from long-run patterns while maintaining a well-controlled risk profile. This includes limiting the extent to which any single feature can influence model predictions, as highly dominant predictors can create concentration risk and lead to unstable portfolio outcomes. In such settings, contribution caps provide a direct mechanism for controlling feature-level influence

while preserving predictive signal. More details will be discussed in Section 5.

3. The Contribution-Cap Framework

3.1. Model Setup

Let $y \in \mathbb{R}^n$ denote the response vector and $X \in \mathbb{R}^{n \times p}$ the predictor matrix, with rows $X_i^\top$ representing observations and columns $X_{\cdot j}$ representing predictors. Consider a generalized linear model with canonical link, under which the linear predictor is

$$\eta = X\beta,$$

The **capnet** package supports model families for which the negative log-likelihood is smooth and convex in η, ensuring that the resulting penalized estimation problem remains convex.

For Gaussian regression with identity link, the negative log-likelihood is

$$\ell(y, \beta, X) = \frac{1}{2}\|y - X\beta\|_2^2.$$

For logistic regression (binomial with logit link),

$$\ell(y, \beta, X) = \sum_{i=1}^{n} \left[\log\left(1 + e^{x_i\beta}\right) - y_i x_i \beta\right].$$

For Poisson regression with log link,

$$\ell(\beta) = \sum_{i=1}^{n} \left(e^{x_i\beta} - y_i x_i \beta\right).$$

For Gamma regression with log link and fixed shape parameter,

$$\ell(\beta) = \sum_{i=1}^{n} \left(y_i e^{-x_i\beta} + x_i \beta\right).$$

These families cover the most commonly used GLMs in applied work and share a convenient structure: the likelihood depends on β only through the linear predictor $X\beta$ and yields a convex optimization landscape. This property allows **capnet** to incorporate additional penalties without altering the fundamental estimation framework.

3.2. Contribution Caps

A distinctive feature of **capnet** is that contribution caps are computed not necessarily on the training matrix X, but on a user-supplied evaluation matrix $Z \in \mathbb{R}^{m \times p}$. Each row of Z represents an evaluation scenario, and each column corresponds to a predictor. This flexibility allows users to regulate predictor influence not only on the training data but also on future or representative environments—an important consideration in applications such as forecasting, risk modeling, or market-timing systems where large contributions may manifest primarily in out-of-sample conditions.

For observation i and predictor j, the realized contribution of predictor j in evaluation context Z is

$$Z_{ij}\beta_j.$$

capnet imposes a soft upper bound on these realized contributions via

$$|Z_{ij}\beta_j| \leq L_j,$$

where $L_j > 0$ is a user-specified cap. Violations of this constraint are penalized through the smooth hinge-squared penalty

$$C(\beta, Z, L) = \sum_{j=1}^{p}\sum_{i=1}^{m}\left(|Z_{ij}\beta_j| - L_j\right)_+^2, \qquad (z)_+ = \max(z, 0).$$

This penalty activates only when the magnitude of a predictor's contribution exceeds its cap and then grows quadratically. Because the hinge-squared function is continuously differentiable, it is directly compatible with gradient-based optimization methods.

The interpretation of L_j is unified across GLM families because all models operate through the linear predictor. In Gaussian models, $Z_{ij}\beta_j$ is an additive contribution to the estimated mean; bounding it directly limits additive influence. In logistic regression, $Z_{ij}\beta_j$ is a change in log-odds, so the cap limits the corresponding odds multiplier to lie within $[e^{-L_j}, e^{L_j}]$. In Poisson and Gamma regression, $Z_{ij}\beta_j$ adjusts the log-mean, yielding a multiplicative bound on the expected count or positive-valued outcome. A single formulation therefore provides additive control in Gaussian models and multiplicative control in log- and logit-link GLMs.

3.3. Penalized Estimation

The **capnet** estimator solves the convex optimization problem

$$\hat{\beta} = \arg\min_{\beta\in\mathbb{R}^p}\left\{\ell(y, \beta, X) + \lambda\left[\frac{1-\alpha}{2}\|\beta\|_2^2 + \alpha\|\beta\|_1\right] + \gamma\, C(\beta, Z, L)\right\} \tag{1}$$

where $\lambda \geq 0$ controls the overall intensity of coefficient regularization, $\alpha \in [0, 1]$ determines the balance between ℓ_1 and ℓ_2 penalties, and $\gamma \geq 0$ controls the strength of the contribution-cap penalty. This formulation mirrors the elastic-net parameterization familiar from Friedman *et al.* (2010) and allows users to employ ridge-like, lasso-like, or hybrid regularization alongside contribution caps.

A key design feature is the explicit separation of the training matrix X and the evaluation matrix Z. When $Z = X$, **capnet** regulates contributions on the training data, recovering the standard setting. When Z differs from X—such as when Z contains representative future market states, stress scenarios, or held-out data—**capnet** ensures that predictor influence remains bounded even in environments that may not be presented in the training sample. This provides a principled mechanism for controlling contribution-level instability or concentration risk in downstream predictive tasks.

The three penalty components interact in complementary ways. The ℓ_1 term enables sparsity, the ℓ_2 term improves numerical stability in the presence of multicollinearity, and the contribution-cap penalty regulates large or volatile contributions. Setting $\gamma = 0$ recovers classical penalized GLMs; setting $\lambda = 0$ yields a pure contribution-regularized model; and

using both penalties allows simultaneous control of coefficient magnitude and predictor influence.

Because the hinge-squared cap penalty is convex and smooth, and the GLM likelihoods considered are convex in the linear predictor, the overall objective remains convex and continuously differentiable except for the nondifferentiability introduced by the ℓ_1 term. This structure is well suited to limited-memory quasi-Newton optimization and provides stable convergence across model families and evaluation matrices.

3.4. Optimization

As discussed above, the negative likelihood function (1) is smooth and convex for all supported GLM families. The only source of nondifferentiability arises from the ℓ_1 term. These properties make the objective well suited to limited-memory quasi-Newton methods. **capnet** uses the **lbfgs** package for optimization Coppola *et al.* (2022). When $\alpha = 0$, the objective function is fully smooth, and standard L-BFGS method is applied. When $\alpha > 0$, the ℓ_1 component introduces non-smoothness at zero, and the optimizer automatically switches to the orthant-wise limited-memory quasi-Newton (OWL-QN) method, which handles the ℓ_1 subgradient by restricting updates to remain within the current orthant, enabling quasi-Newton updates while preserving sparsity-inducing behavior Andrew and Gao (2007).

Calling the **lbfgs** optimizer requires the objective and gradient functions. From (1), we calculate the gradient as

$$\nabla\mathcal{L}(\beta) = \nabla\ell(y,\beta,X) + \lambda\Big[(1-\alpha)\beta + \alpha\,\partial\|\beta\|_1\Big] + \gamma\,\nabla C(\beta,Z,L) \tag{2}$$

The first component is the standard GLM score

$$\nabla\ell(y,\beta,X) = X^\top(\mu - y), \qquad \mu = g^{-1}(X\beta)$$

And for the contribution-cap penalty, the gradient is

$$\frac{\partial C}{\partial \beta_j} = 2\sum_{i=1}^{m}(|Z_{ij}\beta_j| - L_j)_+ \cdot \operatorname{sign}(Z_{ij}\beta_j)Z_{ij}.$$

The hinge-squared form ensures differentiability even at the activation boundary.

Also define the contribution matrix as

$$A = Z\ \operatorname{diag}(\beta) \in \mathbb{R}^{m\times p}$$

$S = \operatorname{sign}(A)$ element-wise, and $V = (|A| - \mathbf{1}L^\top)_+$. The gradient of the contribution-cap penalty can be written in the following matrix form:

$$\nabla_\beta C = 2\Big((V \odot S) \odot Z\Big)^\top \mathbf{1} \in \mathbb{R}^p$$

where $\odot$ is element-wise multiplication, and this is how the gradient calculation is performed in the **capnet** package.

The evaluation matrix Z enters the optimization only through the penalty term. Because $C(\beta, Z, L)$ decomposes additively over predictors, the cost of computing its gradient grows

linearly in the number of rows of Z. This design allows users to specify large or diverse evaluation sets without affecting the GLM likelihood computation.

4. Software Implementation and Example Usage

Function	Purpose
`simulate_capnet_data()`	Generates simulated data for linear regression experiments and examples.
`capnet()`	Fits a contribution-capped elastic net regression model for a fixed set of hyper-parameters.
`walk_capnet()`	Computes a sequence of `capnet` models along a user-specified path of the evaluation dataset.
`cv_capnet()`	Performs cross-validation over a grid of elastic net hyper-parameters to select a preferred model specification.
`coef_path()`	Extracts regression coefficients from a collection of fitted `capnet` models along a hyper-parameter path.
`capnet_violations()`	Identifies and summarizes violations of the soft contribution-cap constraints for a fitted `capnet` model.
`plot_redistribution()`	Visualizes changes in regression coefficients and feature-level contributions between two fitted `capnet` models.

Table 3: Primary user-facing functions provided by the **capnet** package.

This section provides a simple illustration of using the **capnet** package to fit the contribution-capped elastic net regression models. We will mostly consider the linear regression model and then briefly discuss extensions to the logistic, Poisson, and gamma regression cases. The package is available from ... and can be installed and loaded in the usual manner:

```
R> install.packages("capnet")
R> library("capnet")
```

We first load a small simulated dataset, including the training design matrix and response vector, the contribution cap vector, and evaluation design matrix, using the built-in `simulate_capnet_data()` function.

```
R> n <- 100
R> p <- 10
R> m <- 20
R> data <- simulate_capnet_data(n=100,p=10,m=20)
```

```
R> X <- data$X
R> y <- data$y
R> L <- data$L
R> newx <- data1$newx
```

The **capnet** package is mainly used with calls to estimation three functions as shown in Table 3: `capnet()`, `walk_capnet()`, and `cv_capnet()`. The interface is designed to mimic popular R packages for regularized generalized linear models, most notably **glmnet** (Friedman *et al.* 2010). To perform the contribution-capped regression, `capnet()` takes as required inputs, the design matrix `x`, the response vector `y`, and the contribution cap vector `L`. The function also take several optional arguments. The `newx` matrix specifies the evaluation matrix for the feature contributions. The `family` argument specifies the regression model (`"gaussian"`, `"binomial"`, `"poisson"`, or `"gamma"`). `alpha`, `lambda`, and `gamma` controls the elastic net and contribution cap penalty strengths, and `upper.limits` and `lower.limits` vectors specify coefficients box constraints.

By default, the model fits a Gaussian model using the training design matrix `X` as the evaluation matrix and adds no elastic net or contribution cap penalties and no box constraints. Below are sample usages with different combinations of parameters

```
# No regularization, same as running lm()
R> fit1 <- capnet(X, y, L)

# Only elastic net penalties, same as running glmnet()
R> alpha <- 0.5
R> lambda <- 0.1
R> fit2 <- capnet(X, y, L, alpha = alpha, lambda = lambda)

# Adding contribution cap penalties, evaluated on training design matrix
R> gamma <- 1
R> fit3 <- capnet(X, y, L, alpha = alpha, lambda = lambda, gamma = gamma)

# Evaluating capnet on evaluation matrix
R> fit4 <- capnet(X, y, L, newx = newx, alpha = alpha, lambda = lambda,
+    gamma = gamma)

# Adding box constraints
upper.limits <- c(rep(Inf, 5), rep(0, 5))
lower.limits <- c(rep(0, 5), rep(-Inf, 5))
R> fit5 <- capnet(X, y, L, newx = newx, alpha = alpha, lambda = lambda,
+    gamma = gamma, lower.limits = lower.limits, upper.limits = upper.limits)
```

`capnet()` returns an `S3` object, and we have included the `coef` and `predict` functions to display the coefficients and make predictions on new design matrix data.

```
R> coef(fit4)
```

```
                   beta
(Intercept) -0.04415975
X1           0.51001177
X2          -1.71345010
X3           1.02256470
X4          -1.02555103
X5           0.60040569
X6           0.07398728
X7           0.97554411
X8           1.49418615
X9          -0.39978010
X10         -0.02234612

R> predict(fit4, newdata = tail(newx))

            [,1]
[15,] -0.7093457
[16,] -2.7684809
[17,]  2.8157814
[18,] -2.7043587
[19,]  4.6532123
[20,] -0.6632460
```

An accompany function, `coef_path()` is used to visualize the coefficient paths along a sequence of one hyperparameter. The function will return an `S3` object with accompanying `plot()` method to visualize the coefficient paths. The resulting plots from the following example are shown in Figure 1.

```
R> lambdas <- exp(seq(2, -5, length.out = 100))
R> alphas <- seq(0, 1, length.out = 100)

R> path_l <- coef_path(X, y, L, alpha = alpha, gamma = gamma, lambda = lambdas,
+    newx = newx)
R> path_a <- coef_path(X, y, L, alpha = alphas, gamma = gamma, lambda = 0.5,
+    newx = newx)

R> plot(path_l)
R> plot(path_a)
```

The `walk_capnet()` function offers the ability to apply the `capnet()` function iteratively over observations in the evaluation set. This is useful because fitting a **capnet** model over multiple observations can produce an over-penalized model that perform worse in out-of-sample prediction tasks. The walk-forward refitting framework is also useful for time-series tasks, where data arrive sequentially. The `walk_capnet()` functions allows for easy sequential fitting and evaluation along the evaluation path. The interface of `walk_capnet()` is very similar to that of `capnet`, with the exception that it also takes the evaluation dataset `newx` as a required input and has an additional optional argument `walk`, which controls the step size of the iterative fitting procedure.

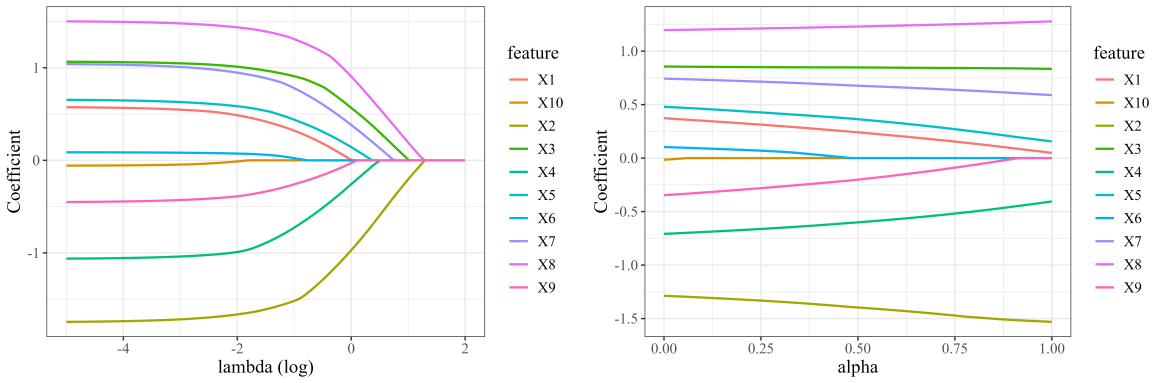

Figure 1: Coefficient paths of **capnet** models fitted on the simulated dataset using the `coef_path` function. The left panel plots estimated coefficients as a function of the elastic net penalty parameter λ. The right panel plots coefficients as a function of the elastic net mixing parameter α. These plots illustrate how coefficient shrinkage and sparsity patterns evolve under elastic net regularization in the presence of contribution caps.

```
R> fit_walk <- walk_capnet(X, y, L, newx, walk = 1, alpha = alpha,
+    lambda = lambda, gamma = gamma)
```

`walk_capnet()` also returns an S3 object and has accompany `coef()` and `predict()` methods to display the sequence of coefficients and predictions with the evaluation set.

```
R> head(coef(walk_fit))

        intercept        X1        X2        X3         X4        X5
[1,] -0.010473888 0.5295727 -1.909948 1.2157910 -1.0642892 0.5980203
[2,] -0.010469909 0.5295734 -1.909942 1.2157780 -1.0642884 0.5980221
[3,] -0.010472161 0.5295688 -1.909946 1.2157802 -1.0642919 0.5980236
[4,]  0.017093792 0.4952672 -1.668442 1.0588963 -0.9992213 0.6257662
[5,] -0.003749633 0.4827738 -1.869136 0.9111731 -1.0221476 0.6174209
[6,] -0.010472971 0.5295687 -1.909944 1.2157801 -1.0642940 0.5980240
             X6       X7       X8         X9         X10
[1,] 0.05092109 1.020880 1.754921 -0.3613088 -0.05495997
[2,] 0.05093191 1.020894 1.754934 -0.3613085 -0.05495241
[3,] 0.05093173 1.020889 1.754937 -0.3613081 -0.05495411
[4,] 0.07342490 1.014856 1.792987 -0.3904655 -0.03321733
[5,] 0.07924715 1.026394 1.780540 -0.3863961 -0.02650655
[6,] 0.05093242 1.020889 1.754933 -0.3613079 -0.05495323

R> tail(predict(walk_fit))

       prediction
[15,] -0.73100631
[16,] -3.24891204
[17,]  3.03570126
```

```
[18,] -2.56909793
[19,]  4.42905610
[20,] -0.05548754
```

The `cv_capnet()` function implements grid-search cross-validation and has a similar signature to `capnet()`. It allows the user to choose the split method (the default is K-fold cross validation) and the loss metrics for measuring prediction or classification accuracy on the held-out sets.[3] `cv_capnet()` allows the user to search for the optimal λ and α values individually or simultaneously.

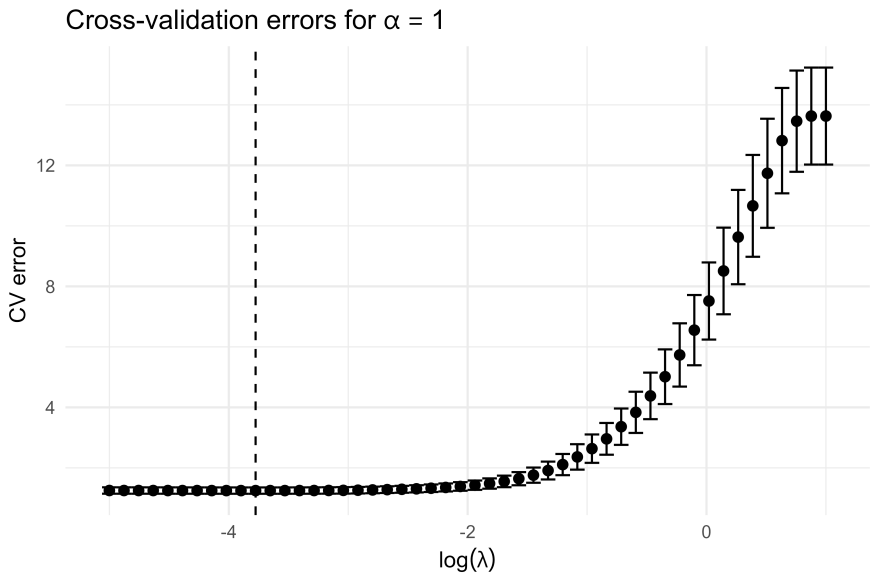

Figure 2: Cross-validation mean-squared error (MSE) of the **capnet** model evaluated over a sequence of λ values on the simulated dataset. Points represent average validation error across folds, and vertical bars denote variability across folds. The dashed vertical line indicates the selected λ minimizing cross-validation error ($\log(\lambda) \approx -3.8$), illustrating the standard model selection procedure implemented in `cv_capnet`.

```
R> gamma <- 0.05

# K-fold grid-search cross-validate on lambda and alpha
R> cv_fit1 <- cv_capnet(X, y, gamma, L, K = 5)

# User-specified cross-validation splits
R> split_id <- rep(1:5, each = 20)
R> cv_fit2 <- cv_capnet(X, y, gamma, L, splits = split_id)

# Grid-search on lambda values only
R> cv_fit3 <- cv_capnet(X, y, gamma, L,
+    lambda = exp(seq(1, -5, length.out = 50)), alpha = 0.5)
```

[3]Currently available metrics include mean-squared error (MSE), out of sample R square, logarithmic loss, brier score, and deviance.

Similar to the implementation by `cv.glmnet()`, `cv_capnet()` uses warm start when searching for `lambda` values to cut computational costs. The `S3 plot()` method displays the cross-validation curve with upper and lower confidence bounds for each λ or α value in the regularization path. Example plots using the simulated data are presented in Figure 2.

```
plot(cv_fit1, alpha = 1)
```

The package also provides diagnostic tools for examining fitted **capnet** models. The function `capnet_violations()` identifies where realized feature contributions exceed the specified contribution caps. Given a fitted **capnet** model, it computes the excess contribution for each feature over the evaluation data. If no new evaluation matrix is supplied, the function uses the same evaluation data that were used when fitting the model. The output includes the row and column indices of all cap violations together with the matrix of excess contributions. The corresponding `print()` method displays this excess matrix in sparse form.

```
R> fit <- capnet(X, y, L, alpha = alpha, lambda = lambda, gamma = 0)

# Default uses the valuation set of the original fit
R> violations1 <- capnet_violations(fit)

# User can specify a new valuation set
R> violations2 <- capnet_violations(fit, newx = tail(newx))
R> print(violations2)

           X1 X2     X3     X4 X5 X6     X7     X8 X9      X10
[15,]       .  . 0.2357      .  .  .      .      .  .        .
[16,] 0.01699  .      .      .  .  .      .      .  .        .
[17,]       .  .      .      .  .  .      .      .  . 0.008612
[18,]       .  .      .      .  .  . 0.3704      .  .        .
[19,]       .  .      .      .  .  .      . 0.8492  .        .
[20,]       .  .      . 0.1989  .  .      . 1.5164  .        .
```

The function `plot_redistribution()` compares two fitted **capnet** models in terms of both their coefficients and their average feature contributions over the evaluation data. It supports two display modes: side-by-side bar charts showing the raw values for each model, and difference plots showing how coefficients and contributions change from one model to the other. Figure 3 shows a side-by-side comparison between an uncapped and a capped model fitted on the simulated dataset.

```
R> newx <- newx[1:5,]
R> fit1 <- capnet(X, y, L, alpha = alpha, lambda = lambda, gamma = 0)
R> fit2 <- capnet(X, y, L, alpha = alpha, lambda = lambda, gamma = 10)

# Side-by-side bar chart comparing feature coefficients and contributions
R> plot_redistribution(fit1, fit2, plot = "delta")

# Bar chart showing changes in feature coefficients and contributions
R> plot_redistribution(fit1, fit2, plot = "raw")
```

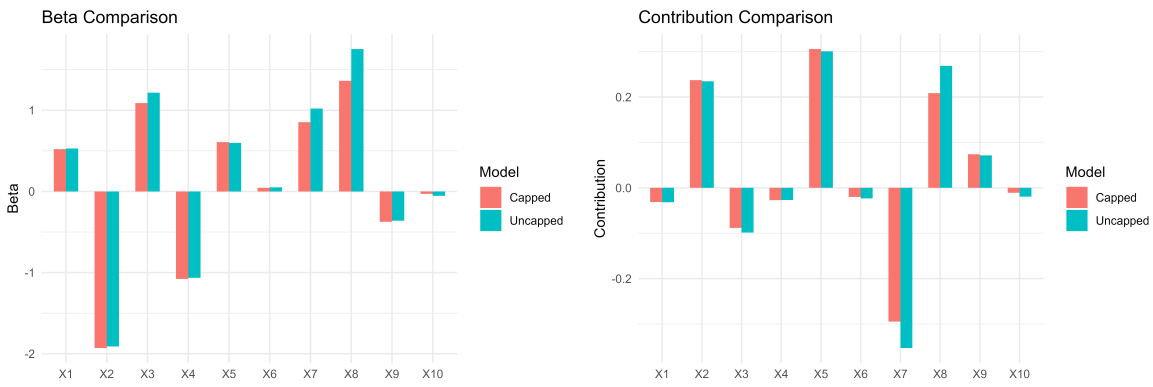

Figure 3: Comparison of coefficient values (left panel) and average feature contributions (right panel) between capped and uncapped **capnet** models fitted on the simulated dataset. Results are generated using `plot_redistribution()` applied to two fitted models with different contribution penalties. Imposing contribution caps reduces the magnitude of coefficients associated with high-impact features (e.g. $X3$, $X7$, $X8$) and redistributes predictive weight to other predictors, demonstrating how **capnet** enforces balanced feature influence.

5. Applications

An important application of **capnet** arises in quantitative finance, where predictor variables frequently exhibit heavy-tailed distributions and episodic extremes. In such environments, standard linear models can produce unstable and economically undesirable predictions, as a small number of extreme feature realizations may dominate the model output. From a portfolio construction perspective, this translates into concentrated exposures and fragile positions that are highly sensitive to outliers. Risk managers may be concerned with these dynamics, as they can lead to unintended bets, amplified drawdowns, and poor tail-risk properties. In practice, portfolio managers often impose explicit constraints—such as position limits, exposure caps, or risk budgets—to control the influence of individual signals. These constraints reflect a preference for diversification not only at the portfolio level but also at the level of predictive signals. However, conventional penalized regression methods (e.g., ridge or elastic net) shrink coefficients uniformly and do not directly control the realized contribution of each feature. As a result, even regularized models can produce predictions dominated by a single extreme predictor when its realized value is large. The contribution-based framework implemented in **capnet** directly addresses this issue by penalizing excessive feature contributions, thereby aligning statistical estimation with practical risk management objectives. The time-series nature of financial data further necessitates rolling estimation procedures, for which the walk-forward implementation `walk_capnet` is particularly well suited.

To illustrate the empirical performance of **capnet**, we use data from the Hull Tactical Market Prediction Kaggle competition. This dataset contains daily observations combining publicly available market data with proprietary features designed to predict excess returns of the S&P 500. Following the variable selection of the competition's baseline model, we construct a backtesting pipeline to evaluate alternative approaches to imposing contribution caps within a linear modeling framework.

Table 4 reports summary statistics for selected variables. While most variables exhibit moderate dispersion, $S5$ and $P12$ display pronounced leptokurtosis. These distributional properties

Variable	Mean	Variance	Excess Kurtosis
S2	0.03	1.04	-0.40
E2	0.51	2.01	-0.16
E3	0.36	2.27	-0.32
P9	0.39	0.15	-1.53
S1	0.24	2.00	1.13
S5	0.03	1.27	14.56
I2	-0.51	1.57	1.66
P8	1.55	0.50	2.87
P10	1.47	0.66	0.34
P12	-0.01	1.21	11.01
P13	0.51	0.08	-1.11

Table 4: Summary statistics of selected variables from the Kaggle dataset. Features $S5$ and $P12$ exhibit substantial positive kurtosis.

imply that extreme realizations occur more frequently than would be expected under Gaussian assumptions. In a standard linear model, such features can exert disproportionate influence on predictions, particularly when extreme values arise out of sample but are not well represented in the training data. This mismatch between in-sample estimation and out-of-sample realization is a key source of model instability and motivates the use of contribution-based regularization.

We consider three modeling approaches: (i) an uncapped baseline, (ii) a post-fit capping method, and (iii) the walk-forward **capnet** approach. At each monthly refit date, models are estimated using a 20-year rolling training window and used to generate predictions for the subsequent month. Given training data `x_train`, `y_train`, test data `x_test`, and a specified contribution cap vector `L`, we estimate `capnet` or `walk_capnet` models with `alpha = 0.5` and a regularization parameter `lambda` selected via `cv.glmnet`. For the post-fit capping approach, we first estimate the uncapped model (equivalently, `capnet` with `gamma = 0`) and then apply contribution caps to the fitted model ex post. While this guarantees that realized contributions satisfy the cap constraint, it does not allow the model to adjust coefficients in response to the constraint during estimation. In contrast, **capnet** incorporates the cap penalty directly into the objective function, allowing the model to reallocate weight across predictors in a manner that balances predictive accuracy with contribution control. The backtest spans January 1, 2020 through February 13, 2026. For the walk-forward specification, we evaluate multiple values of the penalty parameter `gamma` to assess sensitivity to the strength of the cap penalty. Table 5 reports the average out-of-sample R^2 across specifications.

To provide a more granular comparison over time, Figure 4 presents rolling out-of-sample R^2 computed over a 252-day window. The uncapped model (blue) exhibits substantial performance volatility, including large negative deviations that correspond to periods where extreme feature realizations dominate predictions. The post-fit capping approach reduces these downside outcomes and improves robustness during periods of broad underperformance, but at the cost of attenuating strong predictive signals when they are present. In contrast, the walk-forward **capnet** model achieves a more favorable trade-off. By incorporating contribution control during estimation, it selectively dampens the influence of extreme realizations while preserving informative variation in the predictors. This leads to a smoother performance

Method	γ	OOS R^2
Uncapped		0.00706
Post-fit Cap		0.00573
Capnet	0.1	0.00739
Capnet	1	0.00766
Capnet	10	0.00758
Capnet	100	0.00764

Table 5: Average out-of-sample R^2 from the backtest experiment (January 2020 to February 2026) on the Kaggle dataset. The **capnet** models achieve performance comparable to or exceeding the uncapped model while substantially outperforming the post-fit approach, indicating that incorporating contribution constraints during estimation provides a more effective approach in accounting for feature influence.

profile with fewer large drawdowns and more consistentout-of-sample R^2. From a portfolio perspective, this corresponds to improved stability and more reliable signal extraction over time.

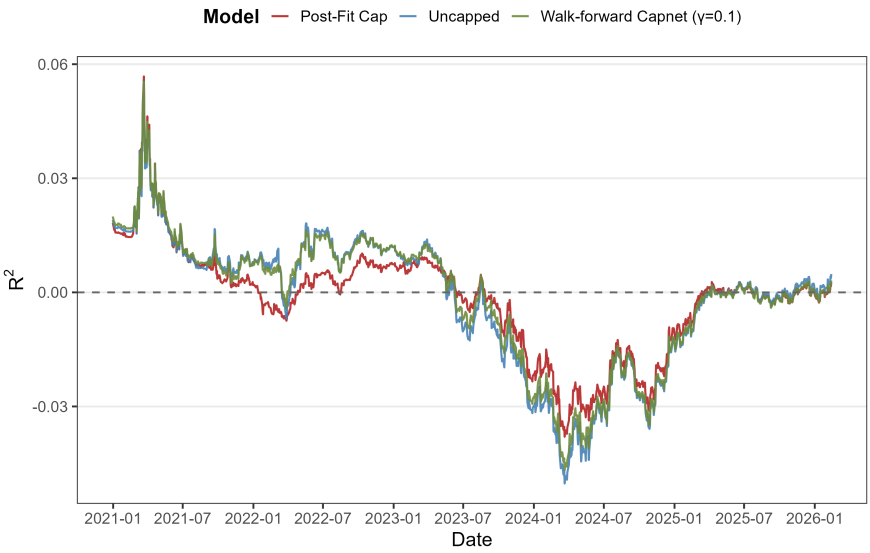

Figure 4: Rolling out-of-sample R^2 computed over a 252-day window for three models: uncapped (blue), post-fit capped (red), and walk-forward **capnet** (green). The uncapped model exhibits substantial performance volatility, including large negative deviations certain periods. The post-fit approach achieves more stable predictive power but attenuates strong signals. The walk-forward **capnet** model achieves a smoother and more stable performance profile.

The performance gains observed for **capnet** can be traced to its ability to regulate feature-level contributions rather than merely shrinking coefficients. In the uncapped model, a small number of predictors have concentrated contributions, resulting in unstable predictions and large drawdowns in performance. Post-fit capping mitigates this effect by truncating contributions ex post, but does not allow the model to adjust its coefficients accordingly. In contrast, **capnet** incorporates contribution constraints directly into the estimation problem, enabling the model to redistribute predictive weight across features. This leads to a more

balanced contribution structure and improved stability over time.

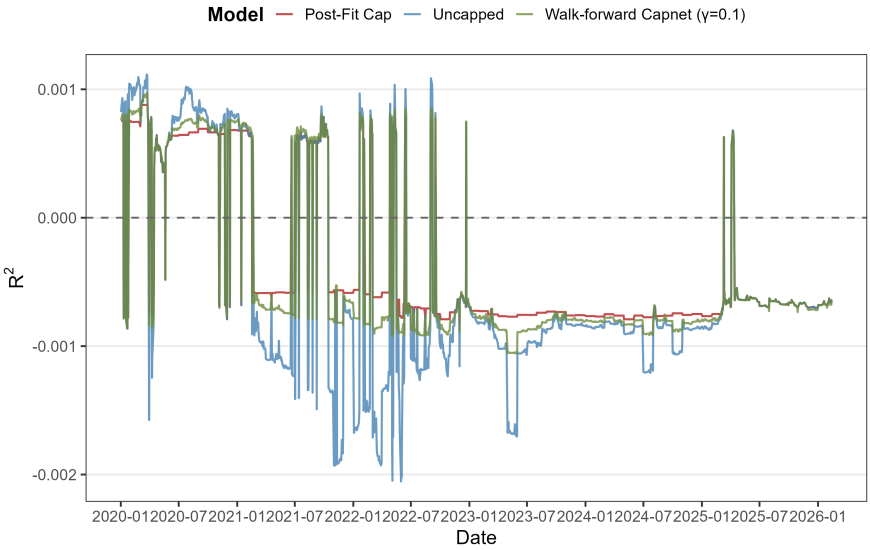

Figure 5: Time series of the maximum absolute feature contribution at each out-of-sample prediction for the uncapped (blue), post-fit capped (red), and walk-forward **capnet** (green) models. The post-fit approach enforces a hard cap, truncating contributions at the specified limit. In contrast, **capnet** imposes a soft constraint via the penalty parameter γ, reducing extreme contributions while preserving flexibility when large contributions are informative.

Figure 5 plots the contribution of the feature with the largest absolute contribution at each prediction point. By construction, the post-fit capping approach enforces a hard constraint, keeping contributions at or within the cap boundary L. In contrast, **capnet** imposes a soft constraint through the penalty parameter γ. For example, with $\gamma = 0.1$, contributions are moderated relative to the uncapped model but are not mechanically truncated at the cap. This allows the model to retain flexibility when extreme contributions are informative, while still discouraging excessive concentration.

To further illustrate how **capnet** redistributes feature contributions, Table 6 reports a snapshot for May 24, 2023. In the uncapped model, features E2 and P8 violate the contribution caps. The post-fit capping approach enforces the constraint mechanically, reducing these contributions without adjusting other coefficients. In contrast, the walk-forward **capnet** model modifies multiple coefficients simultaneously, redistributing contributions across features. This reflects the model's ability to internalize the constraint and solve a constrained optimization problem rather than applying an ex post correction.

From a computational perspective, `capnet` achieves runtime comparable to `glmnet` under the same specification. Table 7 reports the total runtime of the monthly refit backtesting procedure for three methods. All experiments were performed on a machine with an Intel i7-7700 CPU (4 cores, 3.6 GHz) and 32 GB RAM. At each refit date, both the `glmnet` and `capnet` approaches perform a single cross-validation search to select λ, followed by one model fit using the full training sample. In contrast, the `walk_capnet` approach also performs a cross-validation step at each refit date but subsequently refits the model sequentially for each out-of-sample observation. This repeated fitting leads to a higher overall computational cost.

Feature	Uncapped	Post-Fit Cap	Walk-forward **capnet**
S2	−3.439	−3.439	−3.380
E2	−0.140	−0.112	−0.637
E3	0	0	0
P9	0.124	0.124	0.075
S1	−0.156	−0.156	−0.005
S5	0.073	0.073	0.073
I2	−4.863	−4.863	−5.357
P8	−16.07	−7.707	−10.47
P10	0	0	−0.298
P12	0.743	0.743	0.883
P13	−0.641	−0.641	−0.495

Table 6: Feature-level contributions for the uncapped, post-fit capped, and walk-forward **capnet** models on May 24, 2023. All values are scaled by 10^4 for readability. In the uncapped model, features such as $E2$ and $P8$ exceed the contribution ceilings. The post-fit method enforces cap mechanically without adjusting other coefficients. In contrast, the **capnet** model redistributes contributions across multiple features.

Model	Runtime (secs)
`glmnet`	13.06
`capnet`	16.05
`walk_capnet`	58.87

Table 7: Total runtime (in seconds) of the monthly refit backtesting procedure for each method. Both `glmnet` and `capnet` perform a single cross-validation step and model fit at each refit date, while `walk_capnet` additionally refits the model sequentially for each out-of-sample observation, resulting in higher computational cost.

6. Conclusion

This paper introduced **capnet**, an R package for fitting generalized linear models with contribution-capped regularization. The proposed framework extends classical penalized regression methods by directly constraining the realized contributions of predictors to the linear predictor. By shifting the focus of regularization from coefficients to feature-level influence, **capnet** addresses a class of modeling problems in which extreme covariate realizations can disproportionately affect predictions.

The contribution-cap formulation integrates naturally with generalized linear models and yields a convex optimization problem that can be efficiently solved using limited-memory quasi-Newton methods. The ability to specify an evaluation matrix distinct from the training data further enhances flexibility, allowing users to enforce stability not only in-sample but also in stress scenarios or anticipated deployment environments. The package provides a practical implementation of this framework, including tools for model fitting, cross-validation, sequential estimation, and diagnostic analysis, with an interface designed to be consistent with established workflows.

Empirical results demonstrate that contribution-based regularization can improve predictive stability and performance in settings with heavy-tailed predictors. In particular, **capnet**

reduces the dominance of individual predictors by redistributing predictive weight across features, leading to smoother out-of-sample performance and fewer extreme deviations. Compared to post-hoc capping approaches, which enforce constraints after estimation, **capnet** incorporates these constraints directly into the optimization problem, resulting in a more coherent and effective trade-off between robustness and predictive accuracy.

Several limitations and directions for future research remain. The selection of contribution caps requires domain knowledge or heuristic procedures, and developing principled, data-driven approaches for tuning these parameters is an important area for further work. In addition, while the current implementation scales well to moderate evaluation sets, extending the framework to very large or streaming evaluation matrices may require additional computational strategies. Finally, further exploration of the relationship between contribution-based regularization and other robustness frameworks, such as distributionally robust optimization, may yield useful theoretical and practical insights.

Overall, **capnet** expands the toolkit of penalized regression by introducing a new dimension of control over model behavior. By aligning statistical estimation with practical constraints on predictor influence, it provides a flexible and interpretable framework for applications in which stability, robustness, and feature-level interpretability are of central importance.

References

Andrew G, Gao J (2007). "Scalable Training of L1-Regularized Log-Linear Models." In *Association for Computing Machinery (ACM)*, pp. 33–40.

Ben-Tal A, El Ghaoui L, Nemirovski A (2009). *Robust Optimization*. Princeton University Press.

Coppola A, Stewart B, Okazaki N (2022). ***lbfgs**: Limited-memory BFGS Optimization*. R package version 1.2.1.2, URL https://CRAN.R-project.org/package=lbfgs.

Friedman J, Hastie T, Tibshirani R (2010). "Regularization Paths for Generalized Linear Models via Coordinate Descent." *Journal of Statistical Software*, **33**(1), 1–22.

Gan G, Valdez EA (2018). "Fat-Tailed Regression Modeling with Spliced Distributions." *North American Actuarial Journal*, **22**(4), 554–573.

Hoerl AE, Kennard RW (1970). "Ridge regression: Biased estimation for nonorthogonal problems." *Technometrics*, **12**(1), 55–67.

Huber PJ (1973). "Robust Regression: Asymptotics, Conjectures and Monte Carlo." *Annals of Statistics*, **1**, 799–821.

Kosmidis I, Firth D (2019). "Mean and median bias reduction in generalized linear models." *Statistics and Computing*, **29**(4), 709–722.

McCullagh P, Nelder JA (1989). *Generalized Linear Models*. 2 edition. Chapman and Hall, London.

Shi P (2014). "Fat-Tailed Regression Models." In EW Frees, RA Derrig, G Meyers (eds.), *Predictive Modeling Applications in Actuarial Science*, volume 1, pp. 236–259. Cambridge University Press.

Tibshirani R (1996). "Regression shrinkage and selection via the lasso." *Journal of the Royal Statistical Society: Series B*, **58**(1), 267–288.

Wood SN (2017). *Generalized Additive Models: An Introduction with R*. Chapman and Hall/CRC.

Zou H, Hastie T (2005). "Regularization and variable selection via the elastic net." *Journal of the Royal Statistical Society: Series B*, **67**(2), 301–320.